NetShield — Network Intrusion Detection System

Ms. Prapti Soni

Company Name: BharatCares(CSRBOX Group) Bootcamp

Duration: 2 Weeks

Mode: Online

Topic: Cybersecurity

Introduction

NetShield: Network Intrusion Detection System is a cybersecurity project designed to help users identify potentially suspicious activities in network traffic. Network-based attacks, such as port scanning and unusual connection activity, can indicate attempts to discover vulnerabilities or gain unauthorised access to computer systems. NetShield analyzes network traffic records to identify warning signs, such as connections to multiple destination ports, unusually high network-flow counts from a source IP address, and large data transfers within individual flows. It applies predefined security rules to generate alerts, classify their severity, and explain the reasons behind the detection. The project aims to simplify basic network traffic analysis, improve awareness of potential network threats, and support cybersecurity investigation through a simple, user-friendly interface.

Problem Statement

Network-based attacks are a common cybersecurity threat in which attackers scan ports, generate unusual connection activity, or transfer abnormal amounts of data to identify vulnerabilities or compromise computer systems. Identifying these activities can be challenging because suspicious traffic may appear alongside normal network communication. NetShield addresses this problem by analyzing network traffic records using predefined security rules, detecting patterns such as potential port scanning, unusually high flow counts from a source IP address, and large data transfers. It generates alerts, classifies their severity, and explains the reasons behind the detection to help users investigate potentially suspicious activity. The tool aims to simplify network traffic analysis and improve awareness of possible network threats. However, it uses basic rule-based detection, analyzes supplied traffic records rather than capturing live network traffic, and cannot guarantee the detection of every cyberattack.

Objectives

1. To analyze submitted network traffic records in CSV format for potentially suspicious activities.
2. To detect possible port-scanning activity by examining destination-port patterns associated with source IP addresses.
3. To identify unusually high network-flow counts generated by a single source IP address.
4. To detect individual network flows with unusually large amounts of data transferred.
5. To generate alerts, assign severity levels, and explain the reasons behind detected suspicious activities.
6. To provide an interactive dashboard for viewing network traffic records, alerts, and visual summaries.
7. To enable users to filter analysis results and export supported records in CSV format for further investigation.
8. To demonstrate basic network security monitoring through a simple, user-friendly, rule-based application.
9. To support testing and demonstration using sample network traffic data.
10. To improve awareness of potential network threats while recognising the limitations of predefined detection rules.

Tools and Technologies

1. Python: The main programming language used to develop the NetShield application and implement network traffic analysis logic.
2. IBM Bob: An AI-assisted coding tool used to support development, if actually used during the project.
3. Streamlit: Used to create the interactive web dashboard for uploading traffic data, viewing alerts, and exploring analysis results.
4. Pandas: Used to load, process, organize, and analyze network traffic records stored in CSV files.
5. Plotly: Used to create interactive charts and visualizations for presenting network traffic analysis results.
6. CSV Module: Used to support reading or exporting structured traffic data and analysis results in CSV format, where implemented.

Working/Mythology

NetShield follows a rule-based approach to analyze network traffic records and identify potentially suspicious activities.

1. Network Traffic Input: The user uploads a network traffic CSV file or selects the sample traffic dataset provided with the application.

2. Data Preprocessing: Python and Pandas load the traffic records, organize the data, and prepare the required fields for analysis.

3. Rule-Based Detection: The application applies predefined security rules to identify potential port scanning, unusually high flow counts from a source IP address, and unusually large data transfers within individual flows.

4. Alert Generation and Severity Classification: When network traffic matches a detection rule, the application generates an alert with the relevant detection reason and severity information according to the implemented logic.

5. Results and Visualization: The dashboard displays the analyzed traffic records, detected alerts, and available charts to help users understand suspicious patterns.

6. Filtering and Export: Users can filter the displayed results and download supported analysis results in CSV format for further investigation.

7. Sample Data Testing: The application can be tested using synthetic network traffic records to demonstrate how its detection rules identify predefined suspicious patterns.

NetShield provides a simple interface for basic network security monitoring. However, it analyzes supplied network traffic records rather than capturing live network packets, and its rule-based alerts cannot guarantee the identification of every network attack.

Implementation

The NetShield application is implemented in Python using Streamlit. Its main code components are:
1. Application Setup: Configures the Streamlit page, dashboard layout, title, and interface components.

2. Data Preprocessing: The preprocessing.py module loads and prepares network traffic records from CSV files for analysis.

3. Traffic Detection Function: The detector.py module applies predefined rules to identify potentially suspicious network activities, including possible port scanning, unusually high flow counts, and high-byte network flows.

4. Alert Classification: The application generates alerts for traffic matching the implemented rules and provides detection explanations and severity information.

5. Network Traffic Dashboard: The app.py file provides the main interface for selecting traffic data, running the analysis, viewing results, and exploring available charts and summaries.

6. Filtering and CSV Export: The dashboard provides supported controls for filtering analysis results and exporting relevant records in CSV format.

7. Sample Data Generation: The generate_data.py module supports the creation of synthetic network traffic records for testing and demonstration.

8. Dependency Management: The requirements.txt file lists the Python libraries required to install and run the application.

The application uses predefined detection rules rather than a machine-learning model. It analyzes the traffic records supplied by the user and presents potential security alerts through the dashboard. It does not use a permanent database unless one is added separately, and it does not independently capture live network packets.

Output

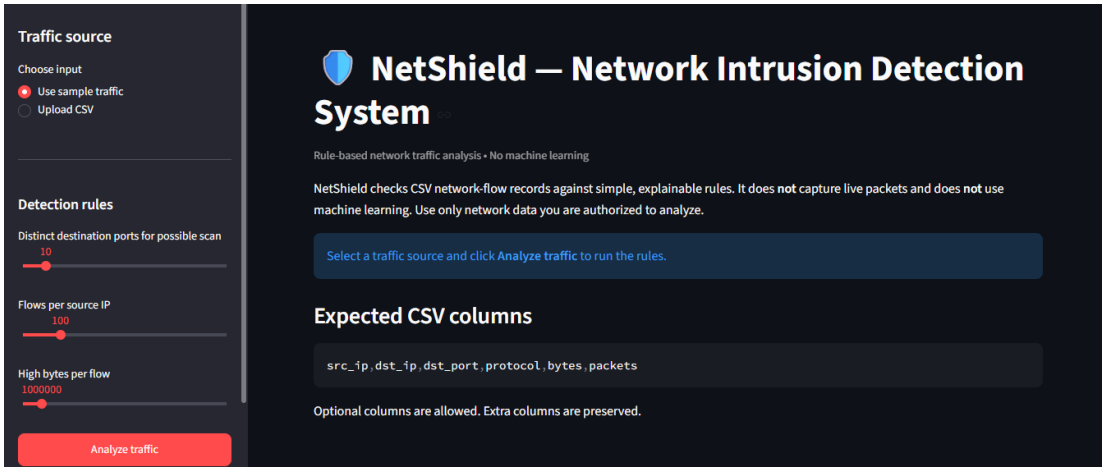


Figure 1: NetShield Landing Interface & Input Configuration

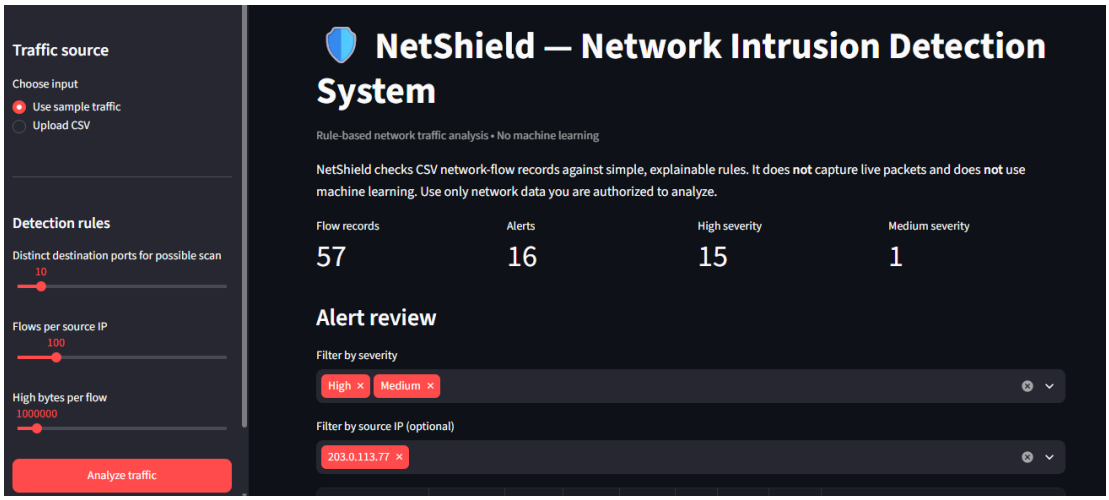


Figure 2: Sample Traffic Analysis Overview & Detection Metrics

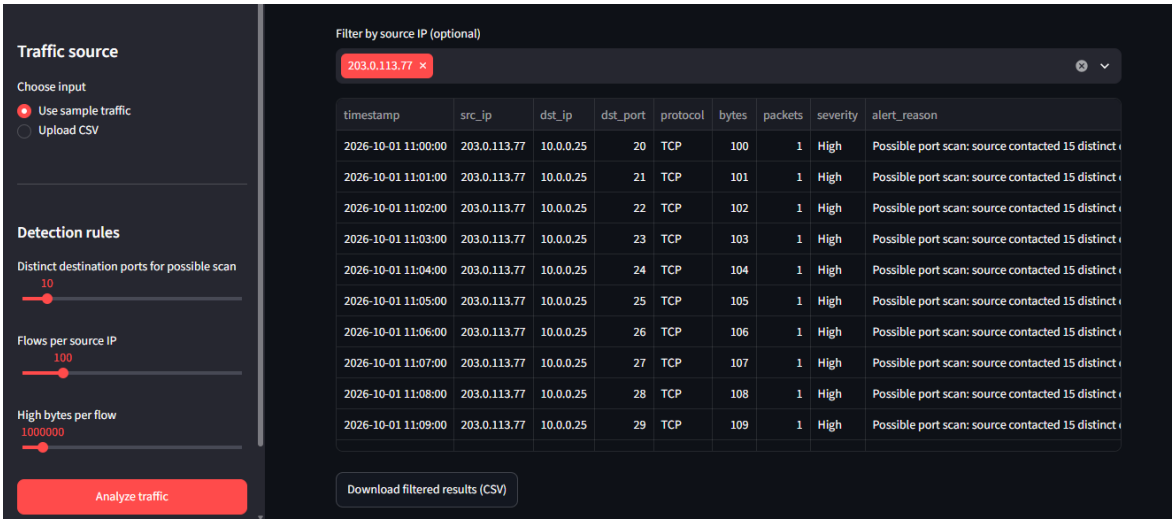


Figure 3: Alert Review Table & Granular Filtering (Sample Traffic)



Figure 4: Traffic Categorization & Protocol Breakdown (Sample Traffic)

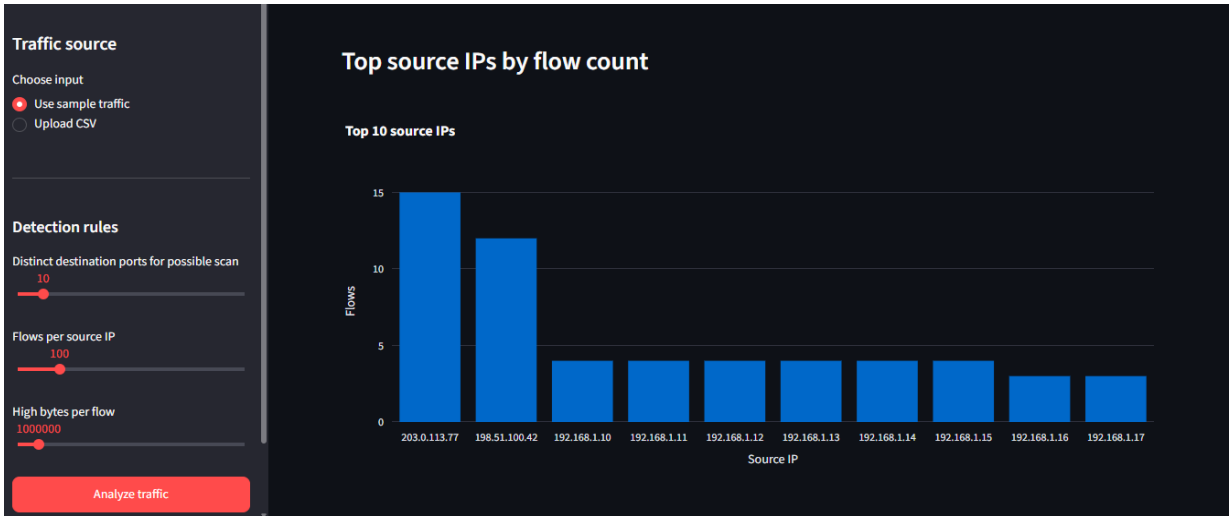


Figure 5: Top Source IP Flow Distribution (Sample Traffic)

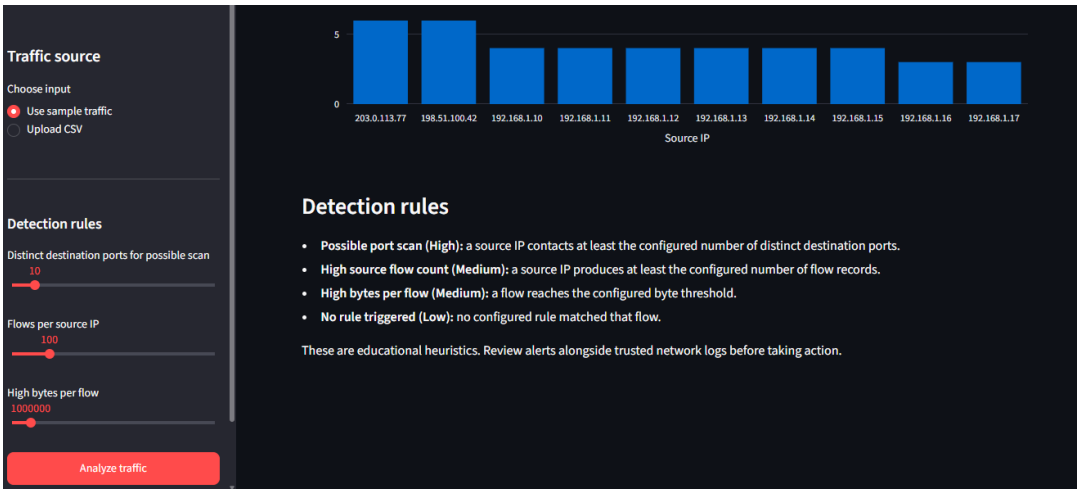


Figure 6: Heuristic Detection Rules & Threat Classification Reference

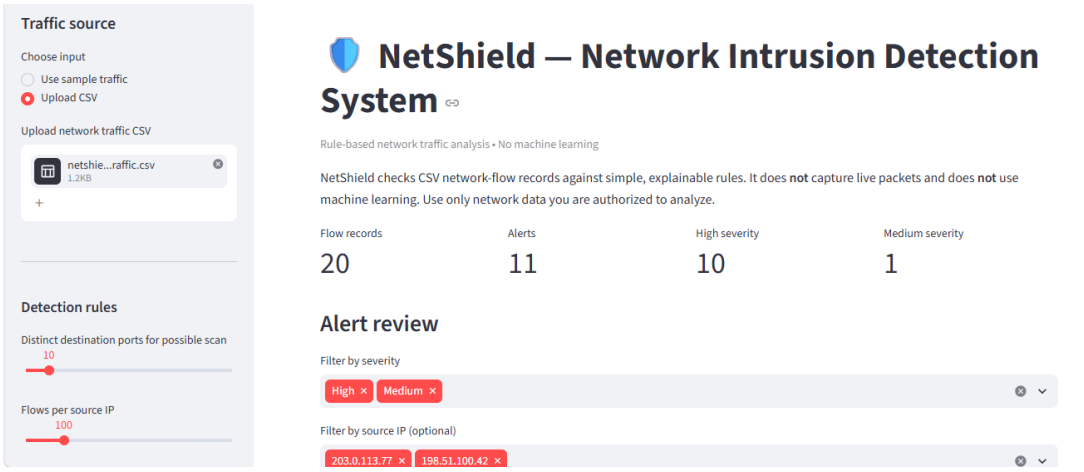


Figure 7: Custom CSV Ingestion & Execution Summary

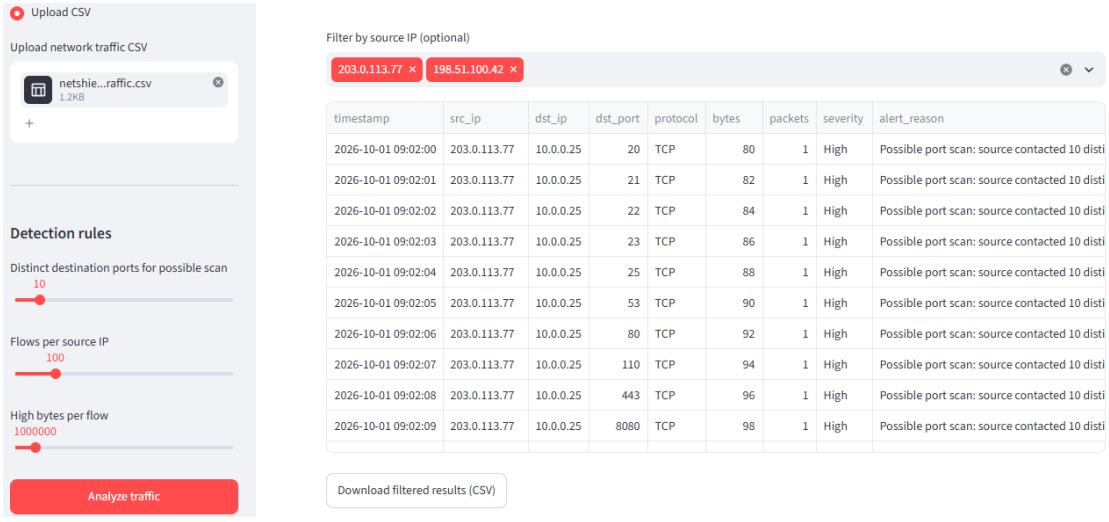


Figure 8: Filtered Threat Log for Uploaded CSV Data



Figure 9: Severity & Protocol Distribution Analysis (Custom CSV)

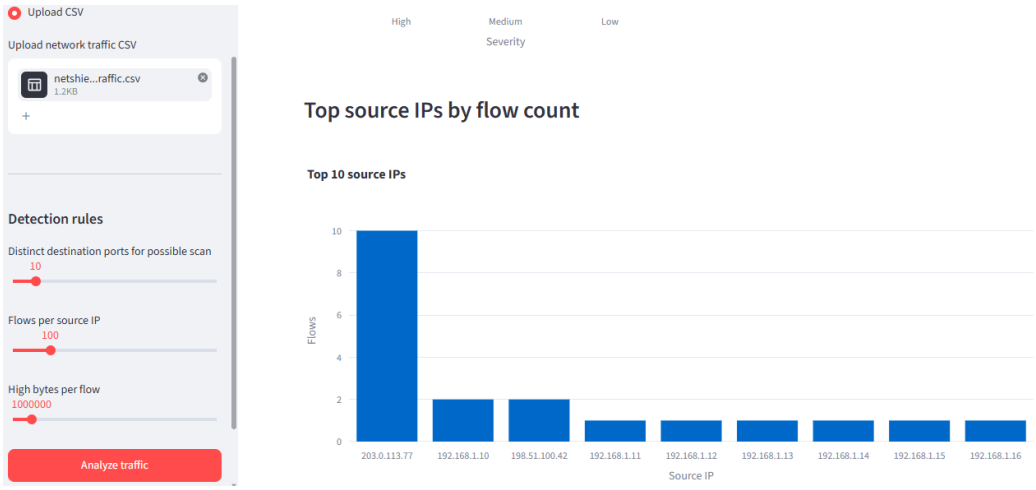


Figure 10: Top Source Talkers & Frequency Analysis (Custom CSV)



Figure 11: Detection Rules Reference & Threat Logic (Custom CSV Upload)

Conclusion

NetShield successfully demonstrates a simple, interactive approach to identifying potentially suspicious network activities through rule-based traffic analysis. It examines network-flow records, detects patterns such as possible port scanning, unusually high flow counts, and large data transfers, and presents alerts with explanations and severity information. Its interactive dashboard and available visualization and export features help users review network traffic and understand potential security concerns.

Limitations: The tool relies on predefined rules and thresholds, so it may miss sophisticated attacks that do not match its detection conditions or flag legitimate network activity as suspicious. Its effectiveness depends on the quality and completeness of the supplied traffic data. It analyzes uploaded or sample network-flow records rather than capturing live network packets, and it does not guarantee the detection of every attack. Therefore, its alerts should be treated as indicators for further investigation rather than definitive proof of a security incident.

Future Scope

1. Real-Time Network Monitoring: Extend NetShield to capture and analyze live network traffic with appropriate permissions and monitoring tools.

2. Advanced Detection Rules: Introduce additional rule-based checks to identify more suspicious connection patterns and potential network attacks.

3. Threat Intelligence Integration: Integrate trusted threat intelligence services to help identify communication with known malicious IP addresses or other suspicious network indicators.

4. Configurable Detection Thresholds: Allow users to adjust detection thresholds according to their network requirements and monitoring needs.

5. Database Integration: Store network traffic summaries, alerts, and investigation history in a database for long-term access and analysis.

6. Advanced Reporting: Generate downloadable PDF reports containing traffic summaries, alert details, severity classifications, and investigation recommendations.

7. Improved Visualization: Add more interactive charts and historical comparisons to help users understand changes in network traffic patterns.

8. Detection Evaluation: Test the system against labeled datasets and documented scenarios to measure detection performance, identify false positives, and discover missed threats.

9. Automated Notifications: Introduce optional notifications when traffic matches specified high-priority detection rules.